

Lean 4における拡張単純型付きラムダ計算の圏論的意味論と操作的健全性

齋藤湊馬

概要

本稿では、原始 Bool 構造を備え、構造を指定したカルテシアン閉圏上に、`STLCextBool` の内在的型付き項と型付き置換の圏論的意味論を構成する。意味論の担体には、型付けを型として保持する内在的構文 `ITerm.Core` を採用する。これにより、変数参照、弱化、束縛子下の持ち上げ、同時置換を型付け証明の消去に依存せず定義できる。

主要結果として、意味論的置換、ラムダ計算の β 簡約と η 規則の意味保存性、原始 Bool の分岐計算規則の意味保存性を証明する。さらに、 β 簡約と Bool 簡約の完全な合同閉包 `CoreStep` と、値呼び関係 `CoreCBVStep` を定義し、前者の多段階簡約と後者の多段階評価が表示を保存することを示す。消去により内在的な `CoreStep` および `CoreCBVStep` を既存の `Step` および `CBVStep` へ写す。さらに、固定した内在的始点から生じる既存の `Step` に対して、内在的な一段階簡約への逆向き持ち上げを示す。結果は Lean 4 で機械検証され、Metatheory ライブラリの Lake ビルドに含まれている。

1 はじめに

型付きラムダ計算の操作的意味論と圏論的意味論の対応は、Curry–Howard–Lambek 対応の具体的な現れである。単純型付きラムダ計算では、型を圏の対象、項を圏の射として解釈し、関数適用とラムダ抽象をカルテシアン閉構造の評価射とカーリー化によって解釈できる [19, 4, 18]。

しかし、Lean 4 でこの対応を機械検証する際には、束縛子、de Bruijn 添字、同時置換、文脈拡張、および操作的簡約の整合性を同時に扱う必要がある。特に、型情報を持たない型付け関係を命題値の関係のまま、型値を返す意味論の担体と直接結びつけると、型付け導出から計算データを取り出す消去が設計上の障害になる。変数束縛表現と置換補題が形式化全体の難易度を左右することは、機械検証されたメタ理論における一般的な設計課題でもある [5, 3]。

本研究では、`STLCextBool` の内在的型付き項を意味論の中心に置く。型情報を持たない項との関係は、消去と操作的関係の型付き持ち上げに分離する。この設計により、意味論の主張を型付き項上で直接述べつつ、既存の `Step` と `CBVStep` を再利用できる。

本稿の貢献は次の通りである。

1. 構造を指定したカルテシアン閉圏と原始 Bool 構造を仮定し、内在的型付き項の表示を定義する。
2. 型付き置換、変数参照、弱化、束縛子下の持ち上げを意味論的に対応づける。
3. 意味論的置換、ラムダ計算の β 簡約・ η 規則、原始 Bool の分岐計算規則を機械検証する。
4. 完全な合同閉包 `CoreStep` と値呼び関係 `CoreCBVStep` の表示保存性を証明し、消去と型付き持ち上げにより既存の操作的関係へ接続する。

2 準備

2.1 型、文脈、内在的型付き項

型は基底型、関数型、および原始 Bool 型からなる。文脈は型のリストであり、本稿では先頭優先規約を用いる。したがって、拡張文脈 $A :: \Gamma$ は、 A の意味対象と Γ の意味対象の直積として解釈する。

Lean 上の型は次の帰納型に対応する。

$$A, B ::= \text{base}(n) \mid A \Rightarrow B \mid \text{Bool} \quad (n \in \mathbb{N}).$$

モデルごとに基底型の解釈 $\text{base}_C : \mathbb{N} \rightarrow \text{Obj}(C)$ を固定し、型解釈を

$$\llbracket \text{base}(n) \rrbracket_C = \text{base}_C(n), \quad \llbracket A \Rightarrow B \rrbracket_C = \llbracket B \rrbracket_C^{\llbracket A \rrbracket_C}, \quad \llbracket \text{Bool} \rrbracket_C = \text{Bool}_C$$

で定める。Lean 対応は `STLCextBool.Ty` と `BoolCCModel.interpretTy` である。

内在的型付き項は判断 $\text{ITerm } \Gamma \vdash A$ を型として持つ。変数は文脈中の参照を満たす証拠、関数適用は関数型と引数型、ラムダ抽象は拡張文脈上の本体、Bool 拡張は `true`、`false`、`if` を持つ。

`ITerm.Core` の消去は内在的型付き項を既存の型情報を持たない `STLCextBool.Term` へ送る。Lean では内在的型付き項から既存の型付け関係への消去の整合性も検証する。

2.1.1 内在的表現を採用する理由

本研究では、意味論の中心となる構文を内在的に型付けされた項として表現する。すなわち、 $\text{ITerm.Core } \Gamma \vdash A$ は、文脈 Γ のもとで型 A を持つ項そのものの型である。項の構成子が型情報を保持するため、型が正しいという性質を別個の命題として構成し、意味関数へ引数として渡す必要がない。例えば関数適用の関数部分が関数型でない項や、`if` の二つの分岐が異なる型を持つ項は、そもそも `ITerm.Core` の要素として構成できない。

これは型情報を持たない構文を捨てるという意味ではない。既存の `STLCextBool.Term` は既存の操作的意味論を表す外在的な構文であり、内在的型付き項からの消去によって接続する。本稿では `ITerm` 上で意味論と型付きの操作的関係を定義し、既存の関係との接続は後段の消去定理と型付き持ち上げ定理に分離する。

2.1.2 項の構成規則

主要な構成規則は次である。

$$\begin{array}{c} \frac{i : \text{Var } \Gamma \vdash A}{\Gamma \vdash_{\text{int}} i : A} \quad \frac{\Gamma \vdash_{\text{int}} f : A \Rightarrow B \quad \Gamma \vdash_{\text{int}} a : A}{\Gamma \vdash_{\text{int}} f a : B} \\[10pt] \frac{A :: \Gamma \vdash_{\text{int}} b : B}{\Gamma \vdash_{\text{int}} \lambda b : A \Rightarrow B} \quad \frac{}{\Gamma \vdash_{\text{int}} \text{true} : \text{Bool}} \quad \frac{}{\Gamma \vdash_{\text{int}} \text{false} : \text{Bool}} \\[10pt] \frac{\Gamma \vdash_{\text{int}} c : \text{Bool} \quad \Gamma \vdash_{\text{int}} t : A \quad \Gamma \vdash_{\text{int}} e : A}{\Gamma \vdash_{\text{int}} \text{if } c \text{ then } t \text{ else } e : A}. \end{array}$$

変数は de Bruijn 式の参照で表される。この表現により、束縛子を越える弱化と置換は、型付きの文脈拡張として扱われる。名前を自然数の距離で置き換える発想は de Bruijn の原典に遡り、型によってスコープ不変条件を保持する表現へ発展している [9, 7, 3]。

2.1.3 具体例：関数適用と恒等関数

文脈を $\Gamma = [A \Rightarrow B, A]$ とする。このとき、先頭変数と二番目の変数は

$$\text{zero} : \text{Var } \Gamma (A \Rightarrow B), \quad \text{succ}(\text{zero}) : \text{Var } \Gamma A$$

である。したがって、

$$t = \text{app}(\text{var}(\text{zero}), \text{var}(\text{succ}(\text{zero})))$$

は $\text{ITerm.Core } \Gamma B$ の項である。恒等関数は

$$\text{id}_A = \text{lam}(\text{var}(\text{zero}))$$

であり、 $\text{ITerm.Core } \Gamma (A \Rightarrow A)$ に属する。このように、型エラーは別途の型検査定理で発見されるのではなく、項の構成自体を Lean が拒否することで排除される。

2.2 構造を指定したカルテシアン閉圏

構造を指定したカルテシアン閉圏モデルは、対象、射、恒等射、合成、終対象、二項直積、指数対象、評価射、カーリー化・非カーリー化、および $\beta \cdot \eta$ 法則を備える構造である。ここで「構造を指定する」とは、カルテシアン閉圏の意味論で通常は同型を除いて定まる構造を、Lean 上で直接参照できるデータとして固定することである [19, 4]。型と文脈の解釈をそれぞれ

$$\llbracket A \rrbracket_C, \quad \llbracket \Gamma \rrbracket_C$$

と書く。空文脈は終対象に、 $A :: \Gamma$ は $\llbracket A \rrbracket_C \times \llbracket \Gamma \rrbracket_C$ に解釈する。

本稿の射の合成は、数学的な通常の右から左への関数合成を記号 “;” で表す。すなわち $f;g$ は「まず f 、次に g 」であり、通常の記法では $g \circ f$ に相当する。この向きを明記することは、Lean の `CCCMModel.comp` と本文中の置換の表示を対応づける上で重要である。

文脈の積の順序も固定する。先頭に A を追加した文脈は

$$\llbracket A :: \Gamma \rrbracket_C = \llbracket A \rrbracket_C \times \llbracket \Gamma \rrbracket_C.$$

そのため先頭変数は第一射影、後続変数は第二射影との合成で解釈される。この順序とカーリー化が要求する引数順の違いを埋める射が、後に現れる $\text{swap}_{A,\Gamma}$ である。

構造を指定したカルテシアン閉圏の公理を、本稿で用いる合成順に書くと、

$$\text{id}_X; f = f, \quad f; \text{id}_Y = f, \quad (f; g); h = f; (g; h)$$

である。直積 $X \times Y$ への対射については、

$$\langle f, g \rangle; \pi_1 = f, \quad \langle f, g \rangle; \pi_2 = g,$$

$$\langle r; \pi_1, r; \pi_2 \rangle = r$$

を用いる。指数対象については、

$$\text{uncurry}(\text{curry}(q)) = q, \quad \text{curry}(\text{uncurry}(r)) = r,$$

および

$$\text{uncurry}(r) = \langle \pi_1; r, \pi_2 \rangle; \text{eval}$$

を用いる。これらが β 簡約・ η 規則の意味保存性の圏論的な基礎になる。

2.3 原始 Bool 構造

原始 Bool 構造は次の指定されたデータからなる。

- Bool 型に対応する対象 Bool_C 。
- 終対象から Bool 対象への真偽値を表す二つの点。
- Bool を判定して任意の対象へ分岐する if 消去子。
- 文脈を表す射に関する if の自然性。
- 真および偽に対する if の計算規則 (β 則)。

Lean の `BoolCCModel` では、これらを `boolObj`、`truePoint`、`falsePoint`、`iteOp`、`ite_true`、`ite_false`、`ite_natural` というフィールドとして保持する。特に

$$\text{iteOp}_{X,Y} : \text{Hom}(X, \text{Bool}_C) \times \text{Hom}(X, Y) \times \text{Hom}(X, Y) \longrightarrow \text{Hom}(X, Y)$$

は同じ文脈対象 X 上の条件と二つの分岐を受け取る。`ite_true` と `ite_false` は分岐計算規則を与え、`ite_natural` は任意の $h : X' \rightarrow X$ に対し、 h との前合成が if 消去子の三引数への前合成と可換であることを要求する。前二者は Bool の根元規則、後者は意味論的置換の if の場合に用いられる。

正確には、 $!_X : X \rightarrow 1$ 、 $\text{truePoint}, \text{falsePoint} : 1 \rightarrow \text{Bool}_C$ とし、任意の $c : X \rightarrow \text{Bool}_C$ 、 $t, e : X \rightarrow A$ に対して

$$\text{ite}_{X,A}(c, t, e) : X \longrightarrow A$$

を指定する。この演算は次の等式を満たす。

$$\text{ite}_{X,A}(!_X; \text{truePoint}, t, e) = t, \quad (1)$$

$$\text{ite}_{X,A}(!_X; \text{falsePoint}, t, e) = e. \quad (2)$$

さらに、 $h : Y \rightarrow X$ に関する自然性は

$$h; \text{ite}_{X,A}(c, t, e) = \text{ite}_{Y,A}(h; c, h; t, h; e) \quad (3)$$

である。両辺はいずれも $Y \rightarrow A$ であり、式 (3) が if の場合の意味論的置換を与える。

注意 2.1. 本稿の定義は原始 Bool を二元余積として構成するものではない。したがって、 $\text{Bool} \cong 1 + 1$ の普遍性は主張しない。

ここで原始 Bool 構造を「指定する」とは、Bool 対象や if 消去子を同型の違いまで商に取らず、モデルごとに具体的な対象と射として選ぶことを意味する。この弱い仮定は、`STLCextBool` に現れる `true`、`false`、`if` の意味を定義し、分岐計算規則を証明するには十分である。一方、Bool の分類性、二元余積との同値性、あるいは自由カルテシアン閉圏としての普遍性は本稿の仮定に含めない。

3 形式化

3.1 項の表示

項の表示は文脈対象から項の型を表す対象への射である。

$$\llbracket t \rrbracket_C : \llbracket \Gamma \rrbracket_C \longrightarrow \llbracket A \rrbracket_C \quad (t : \text{ITerm } \Gamma \ A).$$

以下では $X_\Gamma = \llbracket \Gamma \rrbracket_C$ と略記する。項の表示は構成子ごとに次の再帰方程式で定める。

$$\begin{aligned}\llbracket \text{var}(\text{zero}) \rrbracket_C &= \pi_1, \\ \llbracket \text{var}(\text{succ}(v)) \rrbracket_C &= \pi_2; \llbracket \text{var}(v) \rrbracket_C, \\ \llbracket \text{app}(f, a) \rrbracket_C &= \langle \llbracket f \rrbracket_C, \llbracket a \rrbracket_C \rangle; \text{eval}, \\ \llbracket \text{lam}(b) \rrbracket_C &= \text{curry}(\text{swap}_{A, \Gamma}; \llbracket b \rrbracket_C), \\ \llbracket \text{true} \rrbracket_C &= !_{X_\Gamma}; \text{truePoint}, \\ \llbracket \text{false} \rrbracket_C &= !_{X_\Gamma}; \text{falsePoint}, \\ \llbracket \text{if}(c, t, e) \rrbracket_C &= \text{ite}_{X_\Gamma, \llbracket A \rrbracket_C}(\llbracket c \rrbracket_C, \llbracket t \rrbracket_C, \llbracket e \rrbracket_C).\end{aligned}$$

第二式の第一射影と第二射影は、それぞれ拡張文脈の先頭変数と残りの文脈を表す。ラムダ抽象の場合の swap は、拡張文脈の直積を $\llbracket A \rrbracket_C \times \llbracket \Gamma \rrbracket_C$ としたことにより必要になる。これは単なる記法上の省略ではなく、束縛子下の持ち上げと意味論的置換の証明で実際に用いる射である。

注意 3.1 (表示の型整合性). $t : \text{ITerm.Core } \Gamma \ A$ に対し、 $\llbracket t \rrbracket_C$ は $\text{Hom}_C(\llbracket \Gamma \rrbracket_C, \llbracket A \rrbracket_C)$ に属する。これは独立した事後の定理ではなく、Lean 定義 ITerm.denote の返り値の型そのものによって保証される。

Lean 上の定義名は ITerm.denote である。

3.2 型付き置換

型付き置換 $\sigma : \text{ISubst } \Gamma \ \Delta$ は、 Δ の各変数を Γ 上の項へ送る。束縛子を越える置換は $\sigma.\text{up } A$ で定義し、先頭変数を新しい変数、後続変数を弱化された旧環境へ送る。

置換の表示は

$$\llbracket \sigma \rrbracket_C : \llbracket \Gamma \rrbracket_C \longrightarrow \llbracket \Delta \rrbracket_C$$

を与え、空置換と置換の拡張について

$$\begin{aligned}\llbracket \text{nil} \rrbracket_C &= !_{X_\Gamma} : X_\Gamma \longrightarrow 1, \\ \llbracket \text{cons}(t, \sigma) \rrbracket_C &= \langle \llbracket t \rrbracket_C, \llbracket \sigma \rrbracket_C \rangle : X_\Gamma \longrightarrow \llbracket A \rrbracket_C \times \llbracket \Delta \rrbracket_C\end{aligned}$$

と定める。変数参照定理は自然数添字ではなく、型を保持する変数 $v : \text{Var } \Delta \ A$ について

$$\llbracket \sigma.\text{lookup}(v) \rrbracket_C = \llbracket \sigma \rrbracket_C; \llbracket \text{var}(v) \rrbracket_C$$

を与える。右辺最後の射は、 v が指定する型付き文脈の射影である。

補題 3.2 (項の弱化). $t : \text{ITerm.Core } \Gamma \ B$ を新しい先頭変数 A の下へ弱化すると、

$$\llbracket \text{weaken}_A(t) \rrbracket_C = \pi_2; \llbracket t \rrbracket_C.$$

Lean 対応は $\text{ITerm.denote_weaken_bool}$ である。

補題 3.3 (置換の持ち上げ). $\sigma : \text{ISubst } \Gamma \ \Delta$ に対し、束縛子を越えて持ち上げた $\sigma^\uparrow : \text{ISubst } (A :: \Gamma) \ (A :: \Delta)$ の表示は

$$\llbracket \sigma^\uparrow \rrbracket_C = \langle \pi_1, \pi_2; \llbracket \sigma \rrbracket_C \rangle : \llbracket A \rrbracket_C \times \llbracket \Gamma \rrbracket_C \longrightarrow \llbracket A \rrbracket_C \times \llbracket \Delta \rrbracket_C.$$

Lean 対応は $\text{denoteISubst_up_bool}$ である。

この補題は、束縛子下の証明で繰り返し用いる二つの操作を分離する。項の弱化は新しい変数を無視して旧文脈の表示射を第二射影から引き戻す操作であり、置換の持ち上げは新しい変数を第一射影で保持し、旧変数の置換結果を第二射影から評価する操作である。後者は新しい先頭変数を第一射影で保持し、旧変数の置換結果を第二射影から評価する直積への射である。意味論的置換のラムダ抽象の場合には、この式と明示的な交換射を組み合わせる必要がある。

3.2.1 置換の具体例

$\Delta = [A \Rightarrow B, A]$ に対し、

$$\sigma = \text{cons}(f, \text{cons}(a, \text{nil})) : \text{ISubst } \Gamma \Delta$$

とする。ただし $f : \text{ITerm.Core } \Gamma (A \Rightarrow B)$ と $a : \text{ITerm.Core } \Gamma A$ である。このとき

$$\text{lookup}(\text{zero}, \sigma) = f, \quad \text{lookup}(\text{succ}(\text{zero}), \sigma) = a$$

であり、

$$\text{app}(\text{var}(\text{zero}), \text{var}(\text{succ}(\text{zero})))[\sigma] = \text{app}(f, a)$$

となる。

ラムダ抽象の本体では、新しい束縛変数を置き換えないために置換環境を持ち上げる。

$$(\lambda b)[\sigma] = \lambda(b[\text{up}(A, \sigma)]).$$

ISubst.up は新しい先頭変数をそのまま保持し、旧環境の各成分を文脈拡張に沿って弱化する。この再帰方程式が意味論的置換のラムダ抽象の場合の出発点になる。

4 主定理

本節の主張は二つの層に分かれる。第一に、内在的型付き項上で置換、 β 簡約、 η 規則、原始 Bool の分岐計算規則が表示を保存することを示す。第二に、 β 簡約と Bool 簡約を根元規則とする完全な合同閉包、および値呼びの一段階関係を定義し、前者の多段階簡約と後者の多段階評価の意味保存性を示す。 η 規則は操作的な根元簡約には含めず、表示上成立する等式原理として別に扱う。

型情報を持たない項上の **Step** と **CBVStep** は、それ自体には型情報を持たない。そのため、本稿の直接の保存定理は内在的な **CoreStep** と **CoreCBVStep** に対して述べる。既存の関係との接続は、消去と操作的関係の型付き持ち上げによって別に検証する。この区別により、型情報を持たない項上の表示の一意性を過大に仮定せず、Lean の型システムが保持する内在的な型情報を証明の中で利用できる。

定理 4.1 (意味論的置換). $t : \text{ITerm.Core } \Delta R$ と $\sigma : \text{ISubst } \Gamma \Delta$ に対し、

$$\llbracket t[\sigma] \rrbracket_C = \llbracket \sigma \rrbracket_C ; \llbracket t \rrbracket_C.$$

証明は t に関する構造帰納法である。変数の場合は変数参照定理、関数適用の場合はカルテシアン閉圏の対射と合成の法則、ラムダ抽象の場合は $\sigma.\text{up } A$ の意味論、明示的な交換射、カリー化の前合成則、および自然性を用いる。Bool の if の場合は if 消去子の文脈に関する安定性により閉じる。

Lean 上の定理名は **denote_subst_bool** である。

証明の中心はラムダ抽象の場合である。変数の場合では構文的置換の参照と置換の表示に現れる射影が一致する。関数適用の場合では帰納法の仮定を関数部分と引数部分に適用し、対射、合成、評価射の構成性を用いる。ラムダ抽象の場合では本体を一つ先の文脈へ持ち上げるため、**ISubst.up** の意味論、明示的な交換射、カリー化の前合成則を順に適用する。Bool の if の場合では、if 消去子の文脈に関する安定性が必要になる。

if の場合では帰納法の仮定を条件、真の場合の分岐、偽の場合の分岐の三項に適用する。得られる三つの等式を **ite_natural** に代入すると、 $\llbracket \sigma \rrbracket_C$ との前合成を if 消去子全体の外へ移せる。したがって、この場合を閉じるのは **ite_true/ite_false** ではなく、文脈に関する自然性である。

したがって、この定理は単なる構文的置換の補題ではなく、型付き置換を圏の射として解釈したときの合成則を表している。

ラムダ抽象の場合を射の型まで書くと、 $X_\Gamma = \llbracket \Gamma \rrbracket_C$ 、 $X_\Delta = \llbracket \Delta \rrbracket_C$ 、 $a = \llbracket A \rrbracket_C$ 、 $S = \llbracket \sigma \rrbracket_C : X_\Gamma \rightarrow X_\Delta$ として、

$$\llbracket [\sigma^\dagger] \rrbracket_C = \langle \pi_1, \pi_2; S \rangle : a \times X_\Gamma \rightarrow a \times X_\Delta.$$

一方、カーリー化が受け取る入力順は $X_\Gamma \times a$ であるため、

$$\text{swap}_\Gamma : X_\Gamma \times a \rightarrow a \times X_\Gamma, \quad \text{swap}_\Delta : X_\Delta \times a \rightarrow a \times X_\Delta$$

を明示する。置換後のラムダ本体は

$$\text{curry}(\text{swap}_\Gamma; \llbracket [\sigma^\dagger] \rrbracket_C; \llbracket b \rrbracket_C)$$

と解釈される。交換射の自然性により

$$\text{swap}_\Gamma; \llbracket [\sigma^\dagger] \rrbracket_C = \langle \pi_1; S, \pi_2 \rangle; \text{swap}_\Delta$$

であり、カーリー化の前合成則から

$$\text{curry}(\langle \pi_1; S, \pi_2 \rangle; \text{swap}_\Delta; \llbracket b \rrbracket_C) = S; \text{curry}(\text{swap}_\Delta; \llbracket b \rrbracket_C)$$

を得る。これが意味論的置換の右辺である。Lean では持ち上げの意味を `denoteISubst_up_bool`、変数順序の交換を交換射の自然性、最後のラムダ抽象の再構成をカーリー化の前合成則で処理している。

定理 4.2 (λ 計算の β 簡約の意味保存性). 本体 $b : \mathbf{ITerm.Core} (A :: \Gamma) B$ と引数 $a : \mathbf{ITerm.Core} \Gamma A$ に対し、

$$\llbracket (\lambda b) a \rrbracket_C = \llbracket b[a] \rrbracket_C.$$

これは定理 4.1 と、構造を指定したカルテシアン閉圏の指数対象の β 法則から従う。Lean 対応は `intrinsic_beta_sound` である。

証明では、まず意味論的置換を β 置換に適用する。その結果は本体の表示射を、引数の表示射と恒等射からなる対射で前合成した形になる。具体的に $\text{betaSubst}(a) : \text{ISubst } \Gamma (A :: \Gamma)$ は、先頭変数を a へ、残りの変数を恒等置換へ送る。中間補題 `denote_betaSubst` は

$$\llbracket \text{betaSubst}(a) \rrbracket_C = \langle \llbracket a \rrbracket_C, \text{id}_{X_\Gamma} \rangle$$

を与える。従って意味論的置換から

$$\llbracket b[\text{betaSubst}(a)] \rrbracket_C = \langle \llbracket a \rrbracket_C, \text{id}_{X_\Gamma} \rangle; \llbracket b \rrbracket_C$$

を得る。一方、関数適用の意味論を展開すると評価射とカーリー化の合成が現れる。構造を指定したカルテシアン閉圏の指数対象の β 法則により、この二つの式が一致する。ここで重要なのは、 β 簡約の意味保存性が独立の計算規則ではなく、意味論的置換とカルテシアン閉圏の評価射・カーリー化の法則を接続して得られている点である。

定理 4.3 (λ 計算の η 規則の意味保存性). $f : \mathbf{ITerm.Core} \Gamma (A \Rightarrow B)$ に対して、

$$\llbracket \text{etaExpand}_A(f) \rrbracket_C = \llbracket f \rrbracket_C,$$

ただし

$$\text{etaExpand}_A(f) := \lambda(\text{app}(\text{weaken}_A(f), \text{var}(\text{zero})))$$

である。

証明では、**weaken A f** と新しい変数からなる対射を明示し、非カーリー化、評価射、対射の等式を経由して、構造を指定したカルテシアン閉圏の指数対象の η 法則を適用する。Lean 対応は **intrinsic_eta_sound** と **coreEtaStep_denote** である。

η 展開では、 f を拡張文脈へ弱化し、新しい変数に適用してからラムダ抽象を取る。拡張文脈の第一射影が新しい変数、第二射影が旧文脈に対応するため、対射の計算には交換射が現れる。直積の射影計算と η 法則で交換後の対射を整理し、最後に指数対象の η 法則 $\text{curry}(\text{uncurry}(g)) = g$ を適用することで、展開前の f と同じ表示射を得る。内在的な構成では、 η 展開の新しい変数は構文的に新鮮であるため、通常の付帯条件は構成の型に組み込まれる。

定理 4.4 (原始 Bool の分岐計算規則の意味保存性). 任意の分岐 t, e に対して、

$$\llbracket \text{if true then } t \text{ else } e \rrbracket_C = \llbracket t \rrbracket_C,$$

$$\llbracket \text{if false then } t \text{ else } e \rrbracket_C = \llbracket e \rrbracket_C.$$

Lean 対応は **intrinsic_ite_true_sound** と **intrinsic_ite_false_sound** である。

この二つの定理は原始 Bool 構造に含まれる if 消去子の二つの計算規則を直接適用して得られる。すなわち、構文上の真偽値に対する根元簡約と、意味論上の if の計算規則が対応する。

4.1 内在的な一段階関係

ここで、 $t \rightarrow_r u$ 、 $t \rightarrow_c u$ 、 $t \rightarrow_v u$ をそれぞれ **CoreRootStep**、**CoreStep**、**CoreCBVStep** と書く。根元簡約は次の三規則からなる。

$$\overline{\text{app}(\text{lam}(b), a) \rightarrow_r b[\text{betaSubst}(a)]} \quad \overline{\text{if}(\text{true}, t, e) \rightarrow_r t} \quad \overline{\text{if}(\text{false}, t, e) \rightarrow_r e}.$$

CoreStep はこの根元簡約の完全な合同閉包であり、次の規則で定義する。

$$\begin{array}{c} \frac{t \rightarrow_r u}{t \rightarrow_c u} \quad \frac{f \rightarrow_c f'}{\text{app}(f, a) \rightarrow_c \text{app}(f', a)} \quad \frac{a \rightarrow_c a'}{\text{app}(f, a) \rightarrow_c \text{app}(f, a')} \\[10pt] \frac{b \rightarrow_c b'}{\text{lam}(b) \rightarrow_c \text{lam}(b')} \quad \frac{c \rightarrow_c c'}{\text{if}(c, t, e) \rightarrow_c \text{if}(c', t, e)} \\[10pt] \frac{t \rightarrow_c t'}{\text{if}(c, t, e) \rightarrow_c \text{if}(c, t', e)} \quad \frac{e \rightarrow_c e'}{\text{if}(c, t, e) \rightarrow_c \text{if}(c, t, e')}. \end{array}$$

従って、 $\rightarrow_c$ は特定の評価順序ではなく、ラムダ抽象の内部と選択されない if 分岐を含む全ての項構成子に閉じた簡約関係である。

値の述語 Val は

$$\text{Val}(\text{lam}(b)), \quad \text{Val}(\text{true}), \quad \text{Val}(\text{false})$$

だけから生成される。値呼び関係 $\rightarrow_v$ は次の規則で定義する。

$$\begin{array}{c} \frac{\text{Val}(a)}{\text{app}(\text{lam}(b), a) \rightarrow_v b[\text{betaSubst}(a)]} \quad \frac{f \rightarrow_v f'}{\text{app}(f, a) \rightarrow_v \text{app}(f', a)} \\[10pt] \frac{\text{Val}(f) \quad a \rightarrow_v a'}{\text{app}(f, a) \rightarrow_v \text{app}(f, a')} \quad \frac{c \rightarrow_v c'}{\text{if}(c, t, e) \rightarrow_v \text{if}(c', t, e)} \\[10pt] \overline{\text{if}(\text{true}, t, e) \rightarrow_v t} \quad \overline{\text{if}(\text{false}, t, e) \rightarrow_v e}. \end{array}$$

この定義から $\rightarrow_v \subseteq \rightarrow_c$ が従う。 η 規則は $\rightarrow_r$ 、 $\rightarrow_c$ 、 $\rightarrow_v$ のいずれにも含めない。本稿では η を定理 4.3 の表示上の等式原理として扱う。

定理 4.5 (CoreStep の表示保存性). CoreStep $t u$ ならば、

$$\llbracket t \rrbracket_C = \llbracket u \rrbracket_C.$$

CoreStep は β 簡約、Bool の根元簡約、関数適用・ラムダ抽象・if の合同規則を含む。根元規則の意味保存性を基礎とし、各合同規則の構成子には表示の構成性を適用する。Lean 対応は `coreStep_denote` である。

証明は CoreStep の導出に関する帰納法である。根元の β 簡約の場合は定理 4.2、Bool の根元規則には原始 Bool の二つの意味保存性を用いる。合同規則の場合は、例えば関数適用の左側なら

$$\llbracket f a \rrbracket_C = \langle \llbracket f \rrbracket_C, \llbracket a \rrbracket_C \rangle; \text{eval}$$

を展開し、帰納法の仮定を対射の第一成分へ代入する。他の関数適用、ラムダ抽象、if の合同規則も同じ構成性で処理される。このため、根元の簡約対象だけでなく、任意の構文位置での一段階簡約が意味を保存する。

補題 4.6 (CBV 一段階関係の包含). CoreCBVStep $t u$ ならば CoreStep $t u$ である。

Lean 対応は `coreCBVStep_to_coreStep` である。CoreCBVStep は、 β 規則の引数と関数適用の右側を簡約するときの関数部に値の条件を持つ。また通常の CoreStep と異なり、選択されない分岐を簡約する `iteT/iteF` は持たず、条件だけを `iteC` で評価する。

定理 4.7 (CoreCBVStep の表示保存性). CoreCBVStep $t u$ ならば $\llbracket t \rrbracket_C = \llbracket u \rrbracket_C$ である。

証明では前補題により CoreStep へ送る。したがって、CBV 固有の意味論を別に再構成するのではなく、CoreStep の表示保存性を適用して CoreCBVStep の表示保存性を得る。消去側では内在的な値が既存の値へ送られることを確認し、既存の CBVStep の値制約と接続する。

例えば、 $v = \lambda x.x$ は内在的な CoreValue であり、

$$(\lambda f.f) (\lambda x.x) \longrightarrow_{\text{CBV}} \lambda x.x$$

が許される。CBV の関数適用右側の合同規則では、関数部分が値であることを前提に引数を評価する。これは、

$$(\lambda x.x) ((\lambda y.y) (\lambda z.z)) \longrightarrow_{\text{CBV}} (\lambda x.x) (\lambda z.z)$$

のような評価順序を内在的な関係に反映する。内在的な値を消去すると既存の値になることにより、既存の CBVStep の値条件にも接続できる。

定理 4.8 (多段階簡約の表示保存性). Rewriting.Star CoreStep $t u$ ならば $\llbracket t \rrbracket_C = \llbracket u \rrbracket_C$ である。

定理 4.9 (値呼び多段階評価の表示保存性). Rewriting.Star CoreCBVStep $t u$ ならば $\llbracket t \rrbracket_C = \llbracket u \rrbracket_C$ である。

証明は多段階閉包の反射・末尾帰納法で、一段階ごとの等式を合成する。Lean 上の定理名はそれぞれ `core_step_star_denote` と `core_cbv_star_denote` である。

Rewriting.Star の反射の場合は自明な等式であり、末尾の場合は一段階の保存性と帰納法の仮定を等式の推移律で合成する。CBV の多段階評価は、各 CBV 一段階関係を CoreStep へ送る写像を多段階閉包に持ち上げた後、多段階簡約の定理を適用する。この結果は、CoreStep の任意の有限な簡約列と、CoreCBVStep の任意の有限な評価列が表示を保存することを意味する。

定理 4.10 (消去と型付き持ち上げ). 内在的な CoreStep は消去により既存の `STLCextBool.Step` へ、CoreCBVStep は既存の `STLCextBool.CBVStep` へ送られる。さらに、固定した内在的な始点 t と任意の既存の一段階関係 Step $t.\text{erase } N$ に対し、消去すると N になる型付きの内在的な終点と CoreStep の存在を再構成できる。

この逆向き持ち上げは既存の `CBVStep` 全体については主張しない。Lean 対応は `coreStep_erase`、`inverse_step_lift`、`coreCBVStep_erase` である。

以上を正確に言い換えると、型情報を持たない項に直接表示を定義したのではなく、内在的な項を消去した結果が既存の関係と整合し、その既存の一段階関係を固定した型付きの始点から内在的な経路へ持ち上げられる、という主張である。型情報を持たない項の任意の型付け導出が標準的な内在的型付き項を一意に定めることや、任意の既存の `CBVStep` を逆向きに再構成できることは、本稿の主定理には含めない。

5 形式検証と再現性

検証対象の公開 Lean API は

`Metatheory/Paper2/Bridge.lean`

に置かれ、証明本体は `Metatheory/Paper2/Internal/` に分離されている。公開 API は `Metatheory.lean` からライブラリのビルドに読み込まれる。再現コマンドは次である。

```
cd projects/paper1-metatheory
lake build Metatheory
./scripts/check.ps1
```

Lean 4.24.0 において `lake build Metatheory` は 177 件の処理に成功し、仮置き要素および公理の検査も成功している。Lean 4 自体の設計と実装については de Moura と Ullrich によるシステム論文を参照されたい [10]。

5.1 検証対象の分離

検証は一つの巨大な定理ではなく、構文、置換、意味論、操作的意味論の層に分けて行った。第一に、`ITerm.Core` と `ISubst` により内在的型付き項と型付き置換を定義した。第二に、構造を指定したカルテシアン閉圏と原始 `Bool` 構造に対する表示を定義した。第三に、意味論的置換と各計算規則の意味保存性を示した。最後に、内在的な操作的関係を導入し、`CoreStep`、`CoreCBVStep` およびその多段階閉包を扱った。

この分割の利点は、既存の `HasType` を意味論の計算データとして消費しないことである。型付け命題は消去の整合性の証明に使い、意味論そのものは型情報を保持した内在的型付き項上で定義する。これにより、置換と表示の構造帰納法を Lean の型検査が直接支える形にできる。

5.2 Lean 定理対応表

論文中の主張と Lean ソースの対応をまとめると次のようになる。

`Metatheory.Paper2.Bridge` の定理は、証明本体を複製せず、検証済みの内部定理を一つの入口から再公開する。表 1 の対応は、型情報を持たない項の表示の一意性や、任意の既存の `CBV` 一段階関係の逆向き持ち上げを含意しない。

6 関連研究と位置づけ

6.1 ラムダ計算の圏論的意味論

単純型付きラムダ計算の圏論的意味論では、型を圏の対象、文脈を直積、項を射として解釈し、関数型を指数対象に送る。ラムダ抽象はカーリー化、関数適用は評価射に対応し、 β 法則

論文上の主張	Lean 定理・定義
項の表示	<code>ITerm.denote</code>
消去の型保存	<code>ITerm.Core.erase_hasType</code>
弱化・持ち上げ	<code>ITerm.denote_weaken_bool, denoteISubst_up_bool</code>
意味論的置換	<code>Bridge.semantic_substitution</code>
λ の β 簡約	<code>Bridge.intrinsic_beta_denotation</code>
λ の η 規則	<code>Bridge.intrinsic_eta_denotation</code>
Bool 分岐規則	<code>intrinsic_ite_true_sound, intrinsic_ite_false_sound</code>
CoreStep の表示保存	<code>Bridge.intrinsic_step_denotation</code>
CoreCBVStep の表示保存	<code>Bridge.intrinsic_cbv_step_denotation</code>
多段階簡約・評価の表示保存	<code>Bridge.intrinsic_step_star_denotation, Bridge.intrinsic_cbv_step_star_denotation</code>
既存関係への消去	<code>Bridge.core_step_erase, Bridge.core_cbv_step_erase</code>
固定した始点からの持ち上げ	<code>Bridge.raw_step_intrinsic_lift_exists</code>

表 1: 論文上の主張と Lean 定理の対応

と η 法則は指数対象の普遍性から得られる。この基本構造は型理論とカルテシアン閉圏の対応を扱う標準的な文献に基づく [4, 19, 18, 20]。Seely の研究はこの対応を高階多相ラムダ計算へ拡張しているが、本稿が扱うのは単相の `STLCextBool` である [22]。したがって、本稿はカルテシアン閉圏による解釈そのものの新規性や、構文圏とモデルの完全性を主張しない。焦点は、既存の操作的意味論の API を保ったまま、内在的型付き構文上の意味論・置換・簡約保存性を一つの Lean 開発に接続する点にある。

6.2 束縛構文と置換

本研究の焦点は、この既知の意味論を新たに提案することではなく、de Bruijn 式の内在的型付き項、束縛子を越える型付き置換、明示的な交換射を含む意味論を Lean 4 の型検査器へ接続することにある。特にラムダ抽象の場合では、文脈拡張の直積の順序とカーリー化の引数順を同時に管理する必要があり、紙上では省略されがちな交換射と弱化を証明データとして明示する。

de Bruijn 記法は束縛変数の名前を束縛子までの距離で置き換え、 α 変換の管理を構文表現へ吸収する [9]。Bird と Paterson は入れ子データ型によりスコープの正しさを型へ反映する方法を示した [7]。さらに、型とスコープを安全に扱う構文の体系は、名前変更と置換を一般的な走査として統一する [3]。本稿の `ITerm.Core` と `ISubst` も型とスコープを添字に持つが、汎用構文の体系を構築するのではなく、`STLCextBool` とその既存操作への接続を対象を限定する。

Fiore、Plotkin、Turi の初期意味論は、束縛を持つシグネチャから得られる構文を置換構造を備えた初期モデルとして捉え、意味論的置換の補題を構造的に導く [12]。Fiore と Szamozvancev はこの方向を二階抽象構文と Agda 形式化へ展開している [13]。これらに対し本稿では初期性や自由モデルの普遍性を証明せず、固定された、構造を指定したカルテシアン閉圏モデルへの表示と、既存の **Step**/**CBVStep** への操作的な橋渡しを直接検証する。意味論的置換は初期性から自動的に得るのではなく、`ITerm.Core` の構造帰納法として Lean で証明する。

6.3 操作的意味論と機械検証されたメタ理論

値呼びラムダ計算の基礎的な区別は Plotkin の研究に遡る [21]。型安全性を簡約関係と評価文脈の構文的性質から導く方法は Wright と Felleisen によって体系化された [23]。本稿の目的は進行性や型保存を再証明することではなく、内在的な **CoreStep** と **CoreCBVStep** が圏論的表示を保存し、消去後の一段階が既存の関係に含まれることを示すことである。したがって、操作的型安全性と圏論的健全性は相補的だが別の定理として扱う。

機械検証では、束縛子の表現方法の選択が弱化、置換、帰納原理の証明コストを大きく左右する [5]。本稿は局所無名方式ではなく、de Bruijn 式の内在的な担体を採用した。その代償として、既存の型情報を持たない構文と関係を直接の意味論的担体にせず、消去と操作的関係の型付き持ち上げを別途証明する。Lean 4 は依存型と帰納族を用いてこの分離を表現し、カーネルによる定理検査と Lake による再現可能なビルドを提供する [10]。

6.4 定理証明支援系上の圏論的意味論

Ahrens と Zsidó は、型付き高階構文を内在的に表し、その初期意味論を Coq で形式化した [2]。Buisse と Dybjer は Agda 上で圏論的な依存型理論モデルを扱い [8]、Ahrens、Lumsdaine、Voevodsky は型理論を表す複数の圏論的構造の比較を Coq/UniMath で検証している [1]。これらは構文の初期性または依存型理論のモデル構造を中心とし、本稿のように単純型付きラムダ計算の具体的な一段階関係まで表示保存性へ接続することを主目的とはしない。また Ekici は Coq 上でカルテシアン閉前順序における直観主義命題論の圏論的推論を形式化している [11]。カルテシアン閉構造の健全性を機械検証する点は本稿に近いが、内在的型付き項、型付き置換、値呼び簡約は対象外である。

プログラム言語の表示の意味論について、Benton、Kennedy、Varming は Coq 上の領域理論モデルで、再帰を持つ単純型付き値呼び言語の操作的意味論と表示の意味論の健全性・計算的十分性を検証した [6]。Gadea、Gunther、Pagano も Bool を含む値呼び言語について、内在的表示と外在的表示の整合性および計算的十分性を Coq で扱う [15]。また Forster らは、Coq 上の CBPV について弱簡約・強簡約・等式理論・表示の意味論を一つの開発に統合している [14]。これらは操作的意味論との接続が本稿に近い一方、任意の構造指定付きカルテシアン閉圏ではなく、具体的な領域モデルまたは CBPV 固有のモデルを対象とする。

Lean 4 上では、Ghalayini と Krishnaswami が内在的型付き構文、型付き置換、表示の意味論、意味論的置換を精緻化型の形式化で検証している [16]。同著者らによる SSA の表示の意味論も、型付き構文・置換・等式理論と圏論的モデルの接続を Lean 4 で扱う [17]。前者は任意のカルテシアン閉圏にパラメータ化された STLCextBool の意味論ではなく、後者は SSA を対象とする。

以上の先行研究に対し、本稿は、原始 Bool 構造を備えた任意の構造指定付きカルテシアン閉圏を対象として、STLCextBool の内在的型付き項と置換の表示、意味論的置換、 $\beta \cdot \eta$ ・Bool の健全性、完全な合同閉包と値呼び関係の表示保存性、既存の操作的関係への消去を一つの Lean 4 開発として接続する。調査した範囲では、この組合せを同時に扱う既存の機械検証は確認できなかったが、網羅的な先取権の主張は行わない。

7 考察

本稿の意味論は、型情報を持たない項の型付け導出を表示の計算データとして直接利用しない。代わりに内在的型付き項を担体とし、型情報を持たない構文へは消去と型付き持ち上げで接続する。この分離により、命題値を持つ既存の `HasType` から大きな型への消去を仮定せずに、項に関する意味論的置換を構成できる。

7.1 内在的な担体の効果と代償

内在的な担体を採用すると、表示の定義は型付け証明に依存せず、項の構造に直接帰納できる。関数適用、ラムダ抽象、Bool 条件分岐の各場合で、入力項が正しい型を持つことを再証明する必要がない。一方、既存の操作的意味論との接続コストは増える。既存の一段階関係の終点を内在的型付き項へ戻すには、構文と型情報を同時に再構成する必要があるためである。本研究ではまず内在的な関係上で保存性を証明し、その後、消去によって既存の操作的関係へ接続した。

7.2 操作的意味論との接続範囲

CoreStep には β 簡約、原始 Bool の根元簡約、関数適用、ラムダ抽象、条件分岐の合同規則を含めた。したがって、根元の簡約対象だけでなく、任意の構文位置での一段階簡約について表示保存性を得ている。CBV については、**CoreCBVStep** を **CoreStep** へ送ることで、CBV 固有の意味論を別に構成せず保存性を導いた。ただしこれは CBV の操作的充足性や評価戦略の完全性を証明するものではない。

7.3 既存の操作的関係に関する制限

内在的な一段階関係から既存の **Step** への消去は検証されている。また、固定した内在的な始点から始まる既存の **Step** を内在的な経路へ持ち上げる存在定理も得られている。しかし、型情報を持たない項の任意の型付け導出が一意的な内在的型付き項を定めることや、任意の既存の **CBVStep** が同じ向きに持ち上げられることまでは主張しない。これは Curry 式の表現では関数適用の中間型が消去された項だけから復元できないためである。

7.4 今後の課題

今後の技術的課題は、原始 Bool を有限余積として特徴づけるより強い意味論と、既存の **CBVStep** に対する完全な逆向きの型付き持ち上げである。さらに、自由モデルに関する完全性、再帰型や効果を含む言語への拡張、より一般的な圏論ライブラリとの接続は本稿の範囲外に残る。

8 おわりに

構造を指定したカルテシアン閉圏と原始 Bool 構造の上で、STLCextBool の内在的型付き項と型付き置換に圏論の意味論を与え、意味論的置換、ラムダ計算の β 簡約・ η 規則、原始 Bool の分岐計算規則、および内在的な **CoreStep** の多段階簡約と **CoreCBVStep** の多段階評価の表示保存性を Lean 4 で機械検証した。消去により両関係を既存の **Step** と **CBVStep** へ写し、固定した内在的始点から生じる既存の **Step** については逆向きの型付き持ち上げも示した。型情報を持たない項へ直接表示を定義したわけではなく、任意の既存の **CBVStep** の逆向き持ち上げは主張しない。

参考文献

- [1] Benedikt Ahrens, Peter LeFanu Lumsdaine, and Vladimir Voevodsky. Categorical structures for type theory in univalent foundations. *Logical Methods in Computer Science*, 14(3:18):1–18, 2018.
- [2] Benedikt Ahrens and Julianna Zsidó. Initial semantics for higher-order typed syntax in Coq. *Journal of Formalized Reasoning*, 4(1):25–69, 2011.
- [3] Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. A type- and scope-safe universe of syntaxes with binding: Their semantics and proofs. *Journal of Functional Programming*, 31:e22, 2021.
- [4] Steve Awodey. *Category Theory*. Oxford University Press, 2010.
- [5] Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. Engineering formal metatheory. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '08*, pages 3–15. ACM, 2008.

- [6] Nick Benton, Andrew Kennedy, and Carsten Varming. Some domain theory and denotational semantics in Coq. In *Theorem Proving in Higher Order Logics*, volume 5674 of *Lecture Notes in Computer Science*, pages 115–130. Springer, 2009.
- [7] Richard S. Bird and Ross Paterson. de Bruijn notation as a nested datatype. *Journal of Functional Programming*, 9(1):77–91, 1999.
- [8] Alexandre Buisse and Peter Dybjer. Towards formalizing categorical models of type theory in type theory. *Electronic Notes in Theoretical Computer Science*, 196:137–151, 2008.
- [9] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. *Indagationes Mathematicae (Proceedings)*, 75(5):381–392, 1972.
- [10] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [11] Burak Ekici. Formal categorical reasoning. *Turkish Journal of Mathematics*, 46(4):1538–1552, 2022.
- [12] Marcelo Fiore, Gordon Plotkin, and Daniele Turi. Abstract syntax and variable binding. In *Proceedings of the 14th Annual IEEE Symposium on Logic in Computer Science*, pages 193–202. IEEE Computer Society, 1999.
- [13] Marcelo Fiore and Dmitrij Szamozvancev. Formal metatheory of second-order abstract syntax. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–29, 2022.
- [14] Yannick Forster, Steven Schäfer, Simon Spies, and Kathrin Stark. Call-by-push-value in Coq: Operational, equational, and denotational theory. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 118–131. Association for Computing Machinery, 2019.
- [15] Alejandro Gadea, Emmanuel Gunther, and Miguel Pagano. Mechanization of coherence and adequacy: Being extrinsic extended to subtyping. *Science of Computer Programming*, 197:102512, 2020.
- [16] Jad Elkhaleq Ghalayini and Neel Krishnaswami. Explicit refinement types. *Proceedings of the ACM on Programming Languages*, 7(ICFP):167–194, 2023.
- [17] Jad Elkhaleq Ghalayini and Neel Krishnaswami. The denotational semantics of SSA, 2024.
- [18] Bart Jacobs. *Categorical Logic and Type Theory*, volume 141 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, Amsterdam, 1999.
- [19] Joachim Lambek and Philip J. Scott. *Introduction to Higher Order Categorical Logic*, volume 7 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 1986.
- [20] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [21] Gordon D. Plotkin. Call-by-name, call-by-value and the lambda-calculus. *Theoretical Computer Science*, 1(2):125–159, 1975.
- [22] R. A. G. Seely. Categorical semantics for higher order polymorphic lambda calculus. *The Journal of Symbolic Logic*, 52(4):969–989, 1987.
- [23] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.